DSGN: A Dual-Stream Gated Network for Scalable Simultaneous Surveillance and Containment

Heng Zhang[1,†], Ziyang Zuo[1,†], Guoxiang Zhao[2,*], and Xiaoqiang Ren[1,2,3]

***Abstract*—The simultaneous surveillance and containment (SSC) problem arises in multi-robot applications such as perimeter security and environmental monitoring, where robots must survey a region of interest and coordinate to enclose targets once they are detected. Surveillance favors spatial dispersion for coverage, whereas containment requires target-centric cohesion for enclosure. This conflict makes heuristic switching unstable, and the instability becomes more pronounced as larger robotic networks introduce more inter-agent and agent-target relations. End-to-end DRL reduces reliance on hand-crafted switching, but conventional single-stream policies often mix surveillance and containment cues in one shared feature space that could entangle dispersion-oriented and target-centric signals and weaken task transitions. In this paper, we propose the Dual-Stream Gated Network (DSGN), an architecture designed to address scale-amplified task-transition difficulties by separating surveillance and containment into specialized representation streams and coordinating them through a context-aware gating mechanism. A probabilistic prior further regularizes the gating process, reducing ambiguous intermediate decisions and supporting clearer behavioral transitions under large-scale interaction complexity. Evaluations show that DSGN maintains strong task efficacy and scalability under constant-density cross-scale settings. Additional ablation and phase-specific evaluations show that the proposed dynamic gating mechanism and probabilistic prior support effective task arbitration and reliable mode switching in both pure surveillance and task-switching scenarios. The source code is available at https://github.com/zuozysama/DSGN.**

***Index Terms*—Multi-Robot Systems, Reinforcement Learning, Surveillance Robotic Systems.**

I. INTRODUCTION

As robotic autonomy matures, large-scale multi-robot systems are increasingly deployed across diverse domains, ranging from environmental monitoring and industrial automation to critical infrastructure maintenance [1], [2]. A representative example is autonomous fishing fleets [3], where robots must survey a region of interest and coordinate their response to mobile, non-cooperative entities. These scenarios necessitate two fundamental capabilities: persistent area surveillance to locate multiple targets [4], [5], and coordinated containment operations to physically enclose them [6]. Although searching and containment have each been extensively studied, they are generally formulated separately. The simultaneous surveillance and containment (SSC) problem considered here couples them by requiring a multi-robot system to balance continuous area coverage with timely containment of targets.

Centralized coordination [7] can exploit global information, but its dependence on the global system state makes it prone to the curse of dimensionality and limits its applicability to large-scale robotic networks. Distributed methods [8], [9] mitigate this limitation by allowing robots to coordinate through local observations and inter-robot interactions. Existing distributed methods relevant to the containment component of SSC have primarily focused on target interception and containment. Differential game methods [9] model interactions between pursuers and evaders as coupled optimization problems, but they can be difficult to apply when environment dynamics are hard to model. Bio-inspired heuristic methods [10] use manually designed interaction rules, which are intuitive to implement but can be hard to scale across varying team sizes, environments, and target behaviors. Deep reinforcement learning (DRL) methods [11] have therefore been introduced to learn containment policies directly from interactions, reducing dependence on explicit dynamics modeling and manually specified interaction rules. These studies cover both single-target containment [12] and multi-target settings [13], and recent work [6] has scaled up multi-target coordination to dozens of robots by leveraging attention-based architectures. The minimal capture time in the graph-based pursuit-evasion formulation has been investigated in [14]. However, in practical multi-robot applications such as environmental monitoring [5] and autonomous fishing [3], targets may not be immediately known in the region of interest. This means that containment alone is insufficient and that persistent surveillance is also required to locate targets before coordinated interception or enclosure [4], [5].

Multi-robot surveillance has been widely studied for persistent coverage and target localization. Classical visibility-based pursuit-evasion studies established complete search strategies and searchability conditions in polygonal and curved environments [15], [16], [17]. Multi-robot target search has also addressed scalable planning for moving targets [18], limited sensing and environment geometry [19], and DRL-based mobile-target search [20]. These studies primarily address target search and detection, whereas SSC couples persistent surveillance with post-detection geometric containment. A straightforward way to combine the two behaviors is to formu-

The work was supported in part by the National Natural Science Foundation of China under Grant 62336005, 62403300, 62421004 and 62461160313, and the Shanghai Municipal Commission of Education under Grant 24SG38.

[†]Heng Zhang and Ziyang Zuo contributed equally to this work.

[1]Heng Zhang, Ziyang Zuo, and Xiaoqiang Ren are with the School of Mechatronic Engineering and Automation, Shanghai University, Shanghai 200444, China (e-mail: heng-zhang@shu.edu.cn; zzysama@shu.edu.cn; xqren@shu.edu.cn).

[2]Guoxiang Zhao and Xiaoqiang Ren are with the School of Future Technology, Shanghai University, Shanghai 200444, China (e-mail: gxzhao@shu.edu.cn).

[3]Xiaoqiang Ren is with the National Key Laboratory of Maritime Unmanned Swarm Intelligence, Shanghai University, Shanghai 200444, China.

[*]Corresponding author: Guoxiang Zhao.

late the transition as a switched or hybrid control problem [21], [22]. However, specifying analytical switching conditions for coupled target search and interception in SSC is difficult because the desired behavior depends jointly on coverage status, target observations, robot distribution, and containment geometry.

An end-to-end DRL paradigm offers an alternative to discrete logic for the SSC problem by learning decentralized policies directly from robot local observations [11], [23]. However, conventional unified architectures often encode surveillance and containment cues in a shared feature space, where dispersion-oriented coverage signals and target-centric enclosure signals can introduce conflicting optimization demands [24]. Within this end-to-end paradigm, a commonly used design is to organize a single policy with specialized representation streams and coordinate them through a gating mechanism, as explored in mixture-of-experts [25] and modular reinforcement learning [26]. Fixed or hand-crafted gating mechanisms can be difficult to adapt as observations and task demands change, while naive adaptive gating may incur chattering that could cause task failures.

To address this challenge, we propose the Dual-Stream Gated Network (DSGN). DSGN separates state processing into two specialized streams. A *spatial-explorative stream* reasons about coverage gaps for surveillance, and a *tactical-exploitative stream* computes geometric formations for containment. A context-aware gating mechanism coordinates the two streams to produce a unified policy for surveillance and containment. We further regularize this gating process with a Beta-distribution prior to stabilize switching decisions. Numerical experiments show that DSGN improves the balance between surveillance and containment and maintains performance as the number of agents increases.

The key contributions of this work are as follows: 1) **Task-Disentangled Representation for SSC:** Built on transformer-based relational encoding, DSGN separates surveillance and containment into two task-specific attention streams, which reduce interference between dispersion-oriented coverage and target-centered containment within a unified end-to-end policy. 2) **Robust Distributed Task Arbitration:** The context-aware gating mechanism and Beta-distribution regularization enable robust switching between surveillance and containment. Together, these components support stable and decisive task transitions when target observations are intermittent or task demands change. 3) **Scalable Multi-Robot Coordination Performance:** Across constant-density cross-scale evaluations, DSGN maintains high task efficacy, execution optimality, safe coordination, and stable formation quality as team and environment scales increase. Additional phase-specific evaluations further demonstrate its task-switching and surveillance capabilities.

II. Problem Formulation

We consider a multi-robot system operating in a bounded continuous domain $\mathcal{W} \subset \mathbb{R}^2$. Let $[N] = \{1, \dots, N\}$, $[M] = \{1, \dots, M\}$, $[L] = \{1, \dots, L\}$, and $[K] = \{1, \dots, K\}$ denote the index sets of pursuers, evaders, obstacles, and surveillance nodes, respectively. At time t, the pursuer and evader position sets are $\mathcal{P}^t = \{p_n^t \in \mathcal{W} \mid n \in [N]\}$ and $\mathcal{E}^t = \{e_m^t \in \mathcal{W} \mid m \in [M]\}$. Each static circular obstacle is represented by (b_ℓ, r_ℓ), where $b_\ell \in \mathcal{W}$, $r_\ell > 0$, and $\ell \in [L]$, giving $\mathcal{B} = \{(b_\ell, r_\ell) \mid \ell \in [L]\}$. The fixed surveillance-node set is $\mathcal{C} = \{c_k \in \mathcal{W} \mid k \in [K]\}$, which provides a reference discretization of $\mathcal{W}$ for measuring persistent surveillance. Node $c_k \in \mathcal{C}$ is considered visited at time t if $\|p_n^t - c_k\| \le R_{\text{visit}}$ for some pursuer $n \in [N]$, where $R_{\text{visit}} > 0$ is the node-visit threshold. The idleness $d_k^t \ge 0$ of node c_k records the elapsed time since its most recent visit. The mission requires pursuers to maintain persistent surveillance over $\mathcal{W}$ while containing all evaders.

A. Dec-POMDP Formulation and System Abstraction

We formulate the multi-robot coordination problem as a decentralized partially observable Markov decision process (Dec-POMDP) [27], defined by $\langle [N], \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \mathcal{O}, \gamma \rangle$. Here, $[N]$ is the set of pursuers, $\mathcal{S}$ is the global state space, $\mathcal{A}$ is the discrete action space, $\mathcal{T}$ is the transition function, $\mathcal{R}$ is the team reward function, Ω is the observation space, $\mathcal{O}$ is the observation function, and $\gamma \in [0, 1)$ is the discount factor. At time t, the global state $s^t \in \mathcal{S}$ consists of pursuer states $\{x_n^t\}_{n\in[N]}$, evader states $\{y_m^t\}_{m\in[M]}$, static obstacles $\mathcal{B}$, and node idleness values $\{d_k^t\}_{k\in[K]}$. The pursuer state is $x_n^t = (p_n^t, v_n^t, \psi_n^t)$, where v_n^t and ψ_n^t denote the linear speed and heading of pursuer n. The evader state is $y_m^t = (e_m^t, u_m^t, \phi_m^t)$, where u_m^t and ϕ_m^t denote the linear speed and heading of evader m. Each pursuer selects a discrete kinematic command $a_n^t = (a_{\text{acc},n}^t, \omega_n^t) \in \mathcal{A}$, where $a_{\text{acc},n}^t$ and ω_n^t denote the linear acceleration and angular velocity commands, respectively. With time step Δt, pursuer n follows the kinematic update $p_n^{t+1} = p_n^t + v_n^t \Delta t [\cos\psi_n^t, \sin\psi_n^t]^\top$, $v_n^{t+1} = v_n^t + a_{\text{acc},n}^t \Delta t$, and $\psi_n^{t+1} = \psi_n^t + \omega_n^t \Delta t$. Due to decentralized execution, pursuer $n \in [N]$ receives a local observation $o_n^t = \mathcal{O}(s^t, n) \in \Omega$, including its own state and nearby teammates, evaders, and obstacles within the sensing radius $R_{\text{sense}} > 0$. The locally visible evader set is $\mathcal{E}_{\text{vis}}^{n,t} = \{m \in [M] \mid \|p_n^t - e_m^t\| \le R_{\text{sense}}\}$. When evaders are detected by any pursuer, their detections are shared within the team to form the aggregated observed evader set $\mathcal{E}_{\text{obs}}^t = \bigcup_{n\in[N]} \mathcal{E}_{\text{vis}}^{n,t}$. Relative position, distance, bearing, and relative velocity are treated as deterministic quantities derived from these observations.

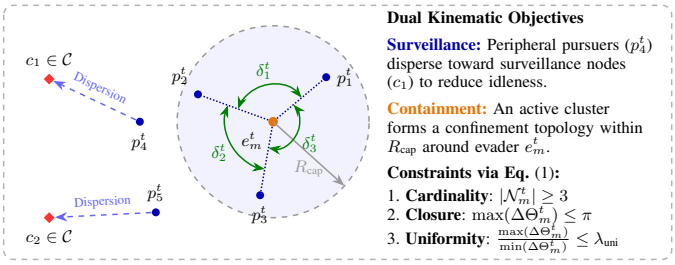


Fig. 1. Visualizing the dual kinematic objectives.

B. Kinematic Objectives of the SSC Problem

The SSC problem couples two kinematically different objectives, as illustrated in Fig. 1. The surveillance objective requires pursuers to disperse over $\mathcal{W}$ to reduce node idleness,

whereas the containment objective requires pursuers to contract around evaders and form target-centered confinement. For evader $m \in [M]$, the active containment cluster at time t is $\mathcal{N}_m^t = \{n \in [N] \mid \|p_n^t - e_m^t\| \le R_{\text{cap}}\}$, where $R_{\text{cap}} > 0$ is the capture radius. For each pursuer $n \in \mathcal{N}_m^t$, let θ_n^t denote its bearing angle with respect to evader m. After sorting these bearings in ascending order, let $\Delta\Theta_m^t = \{\delta_1^t, \dots, \delta_{|\mathcal{N}_m^t|}^t\}$ be the set of adjacent angular gaps, where $\delta_j^t = \theta_{j+1}^t - \theta_j^t$ for $j = 1, \dots, |\mathcal{N}_m^t|$, and $\theta_{|\mathcal{N}_m^t|+1}^t = \theta_1^t + 2\pi$. Evader m is contained at time t if the active cluster satisfies:

$$\mathbb{I}_{\text{cap}}(e_m^t) \iff \left(|\mathcal{N}_m^t| \ge 3\right) \wedge \left(\max(\Delta\Theta_m^t) \le \pi\right) \wedge \left(\frac{\max(\Delta\Theta_m^t)}{\min(\Delta\Theta_m^t)} \le \lambda_{\text{uni}}\right), \tag{1}$$

where $\lambda_{\text{uni}} \ge 1$ is a uniformity coefficient that enforces a balanced angular distribution, with $\lambda_{\text{uni}} = 1$ corresponding to equal angular dispersion among pursuers. This condition ensures both closure of the evader escape cone and stability of the multi-robot containment formation. The objective of SSC is therefore to maintain low surveillance idleness while satisfying the containment condition in Eq. (1) for all evaders. Since this task objective cannot be directly optimized by reinforcement learning, the learned policy is trained through the expected discounted return $J(\pi) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t \mathcal{R}_t]$, where $\mathcal{R}_t$ reflects the SSC requirements.

III. Approach

To address the structural incompatibility between surveillance and containment outlined in Section II, we propose the Dual-Stream Gated Network. DSGN decouples the state representation process into spatial-explorative reasoning for area surveillance and tactical-exploitative coordination for target containment. This architectural inductive bias allows each pursuer to preserve task-specific representations while a context-aware gating mechanism dynamically arbitrates the two streams to produce a unified decentralized policy.

A. Observation and Input Representation

As defined in Section II, each pursuer $n \in [N]$ receives a local observation $o_n^t = \mathcal{O}(s^t, n) \in \Omega$. For network processing, o_n^t is converted into a fixed-size structured input sequence X_n^t, consisting of an ego token and external-entity tokens. The ego token encodes $x_n^t = (p_n^t, v_n^t, \psi_n^t)$, the minimum distance to locally perceived obstacles, and the evader-observation flag $M_{\text{vis}}^t = \mathbb{I}(|\mathcal{E}_{\text{obs}}^t| > 0)$. External entities include locally perceived teammates, observed evaders in $\mathcal{E}_{\text{obs}}^t$, visible obstacles, and surveillance nodes $c_k \in \mathcal{C}$. For each external entity ξ at position z_ξ^t, we use the ego-centric geometric core $[z_\xi^t - p_n^t, \|z_\xi^t - p_n^t\|, \text{bear}(z_\xi^t - p_n^t, \psi_n^t)]$, where $\text{bear}(\cdot)$ is the bearing relative to pursuer n's heading. Entity-specific attributes are appended compactly: relative velocity for dynamic entities, engagement intent for teammates, heading-orientation error for evaders, radius r_ℓ for obstacles, and normalized idleness $\bar{d}_k^t = \text{Norm}(d_k^t)$ for surveillance nodes. Padding and an entity-validity mask $M_{\text{valid}}^{n,t}$ handle varying entity cardinalities. All these features are deterministic functions of the observations defined in Section II, rather than additional problem assumptions.

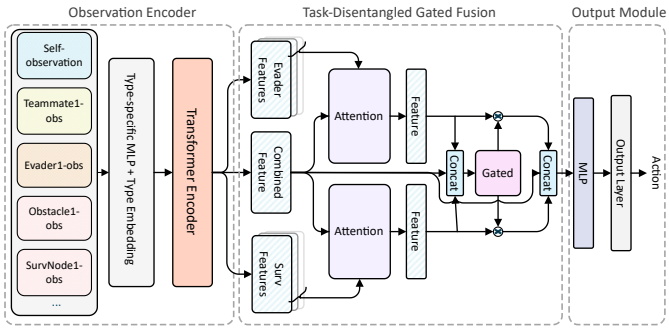


Fig. 2. The proposed DSGN architecture. Type-specific observation features are encoded and fused through dual-stream attention and a vector gate to produce actions.

B. Network Architecture

The proposed DSGN, illustrated in Fig. 2, is an end-to-end pipeline that maps the structured input sequence X_n^t to a discrete kinematic command $a_n^t \in \mathcal{A}$. Instead of prematurely compressing surveillance and containment cues into a single shared representation, DSGN processes heterogeneous observations through four stages: *heterogeneous feature projection*, *relational context encoding*, *task-disentangled gated fusion*, and *distributional action inference*. This design first aligns different entity types into a common latent space, then reasons about their spatial relations, and finally arbitrates two task-specific contexts through a learnable gate.

1) Heterogeneous Feature Projection: The input sequence X_n^t contains heterogeneous entities, including the ego token, teammates, observed evaders, visible obstacles, and surveillance nodes. These entities carry different semantics and feature dimensions: for example, a surveillance node mainly reflects coverage urgency through idleness, whereas an observed evader provides target-centric geometric cues. To avoid mixing such semantics at the raw-feature level, we use type-specific multi-layer perceptrons to project each token into a common latent space $\mathbb{R}^d$, where d denotes the latent feature dimension. A learnable type embedding E_{type} is added to preserve categorical identity after projection, and the validity mask $M_{\text{valid}}^{n,t}$ prevents padded entities from contributing to attention. This projection stage provides a unified but type-aware representation for subsequent relational reasoning.

2) Relational Context Encoding: After feature projection, each pursuer must reason not only about individual entities but also about their spatial and task relations. For example, a neglected surveillance node is more informative when nearby teammates are absent, and an observed evader is more actionable when enough pursuers are already forming a partial enclosure. We therefore employ a transformer encoder to capture high-order dependencies among the projected entity tokens. It takes the type-aware sequence $E_{\text{in}}^{n,t} + E_{\text{type}}$ and validity mask $M_{\text{valid}}^{n,t}$ as input and produces $E_{\text{ctx}}^{n,t}$, where $E_{\text{in}}^{n,t}$ denotes the projected input sequence. This encoder aggregates local scene geometry, teammate distribution, evader observations, obstacle constraints, and surveillance-node urgency into context-aware entity representations. The updated ego feature $h_{\text{ego}}^{n,t}$, extracted from the ego-token position in $E_{\text{ctx}}^{n,t}$, serves as the central query

for dual-stream decision-making.

3) Task-Disentangled Gated Fusion: Given the surveillance–containment conflict discussed in Section II, DSGN uses two specialized attention streams to maintain task-specific contexts before fusion.

Spatial-Explorative Surveillance Stream: This stream focuses on coverage-oriented reasoning. It uses the ego feature $h_{\text{ego}}^{n,t}$ to query encoded surveillance-node features, producing a surveillance context $h_{\text{sur}}^{n,t}$. Because surveillance-node tokens contain idleness information and spatial relations to the current team distribution, $h_{\text{sur}}^{n,t}$ highlights neglected regions and encourages dispersion toward coverage gaps. **Tactical-Exploitative Containment Stream:** This stream focuses on target-centered coordination. It uses $h_{\text{ego}}^{n,t}$ to query encoded observed-evader and teammate features, producing a containment context $h_{\text{con}}^{n,t}$. This representation captures relative target geometry, teammate support, and local formation requirements. When no evader is observed, the stream still processes teammate configurations, which helps preserve tactical readiness without forcing premature containment behavior. **Dynamic Arbitration via Vector-wise Gating:** To balance the two task contexts, DSGN generates a vector gate $g_n^t \in (0,1)^d$ by applying the sigmoid function $\sigma(\cdot)$ to MLP_{gate} of the concatenated features $h_{\text{ego}}^{n,t} \oplus h_{\text{sur}}^{n,t} \oplus h_{\text{con}}^{n,t}$, where $\oplus$ denotes concatenation. Each gate dimension modulates one latent channel. The fused representation $h_{\text{fused}}^{n,t}$ is the projection of the concatenated features $h_{\text{ego}}^{n,t}$, $g_n^t \odot h_{\text{con}}^{n,t}$, and $(1-g_n^t) \odot h_{\text{sur}}^{n,t}$, where $\odot$ denotes the Hadamard product. Large gate values emphasize containment-related channels, whereas small values emphasize surveillance-related channels. Unlike a scalar switch, this vector-wise gate allows different latent dimensions to encode different aspects of the current situation, such as coverage urgency, target proximity, teammate support, and obstacle risk. As a result, the policy can maintain surveillance behavior while gradually committing to containment when target evidence becomes reliable.

4) Distributional Action Inference: The fused representation $h_{\text{fused}}^{n,t}$ is mapped to kinematic decisions using an implicit quantile network (IQN) [28]. Instead of estimating only a scalar action value, IQN approximates the return distribution $Z_\theta(o_n^t, a)$ for each action $a \in \mathcal{A}$. This is useful in SSC because partial observability and multi-robot interactions can make the outcome of the same action highly variable across local contexts.

C. Multi-Objective Reward Design

Section II defines the SSC objective as maintaining persistent surveillance while satisfying the containment condition in Eq. (1). Since this objective cannot be directly used as a reinforcement-learning signal, we design $\mathcal{R}_t$ as a shaped reward that reflects surveillance, containment, safety, and efficiency requirements. This design converts the task-level SSC objective into dense learning signals for decentralized policy optimization. For surveillance, pursuer n receives the arrival reward $r_{\text{arr}}^{n,t} = \psi_{\text{arr}} \sum_{k\in[K]} \bar{d}_k^t \mathbb{I}_{\text{visit}}^{n,k,t}$ when visiting urgent nodes, where $\psi_{\text{arr}} > 0$, $\bar{d}_k^t = \text{Norm}(d_k^t)$, and $\mathbb{I}_{\text{visit}}^{n,k,t} = \mathbb{I}(\|p_n^t - c_k\| \le R_{\text{visit}})$. Because arrival rewards are sparse, we add a potential-based shaping term toward the most urgent node $k^*(t) = \arg\max_{k\in[K]} d_k^t$. The shaping reward is $r_{\text{shape}}^{n,t} = \psi_{\text{shape}}(\|p_n^{t-1} - c_{k^*(t)}\| - \|p_n^t - c_{k^*(t)}\|)$, where $\psi_{\text{shape}} > 0$. This follows policy-invariant reward shaping [29] to get a smooth gradient toward neglected surveillance regions.

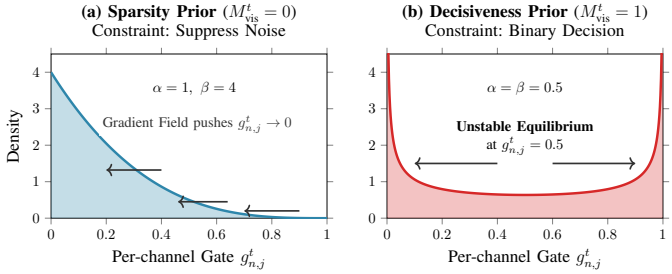


Fig. 3. State-dependent Beta priors for gate regularization.

For containment guidance, let $d_{\min}^{n,t} = \min_{m\in\mathcal{E}_{\text{obs}}^t} \|p_n^t - e_m^t\|$ denote the distance to the closest observed evader when $\mathcal{E}_{\text{obs}}^t \neq \emptyset$. To reduce overcrowding around the same target, each evader admits only the N_{eng} closest pursuers into its engagement set, where N_{eng} is a fixed parameter. The engagement indicator $\mathbb{I}_{\text{eng}}^{n,t}$ is one if pursuer n is non-redundant and $d_{\min}^{n,t} < d_{\text{react}}$, with $\mathbb{I}_{\text{sur}}^{n,t} = 1 - \mathbb{I}_{\text{eng}}^{n,t}$. When no evader is observed, $\mathbb{I}_{\text{eng}}^{n,t} = 0$, $\mathbb{I}_{\text{sur}}^{n,t} = 1$, and $r_{\text{guide}}^{n,t} = 0$. If $d_{\min}^{n,t} < d_{\text{buf}}$, the containment guidance reward is $r_{\text{guide}}^{n,t} = -\psi_{\text{rep}}(d_{\text{buf}} - d_{\min}^{n,t})$; otherwise, it is $r_{\text{guide}}^{n,t} = \psi_{\text{app}}(d_{\min}^{n,t-1} - d_{\min}^{n,t})$, where $d_{\text{react}}, d_{\text{buf}}, \psi_{\text{rep}}, \psi_{\text{app}} > 0$. The first term prevents unsafe proximity to evaders, while the second provides a continuous approach gradient toward containment formation.

Finally, $r_{\text{col}}^{n,t}$ penalizes dangerous proximity to teammates or obstacles, $r_{\text{con}}^{n,t}$ rewards satisfying Eq. (1), and $r_{\text{win}}^{n,t}$ rewards full task completion. We also include a small step penalty $r_{\text{time}}^{n,t} < 0$ to encourage efficient task completion. The total reward $r_{\text{total}}^{n,t}$ sums $r_{\text{arr}}^{n,t}$, the gated terms $\mathbb{I}_{\text{sur}}^{n,t} r_{\text{shape}}^{n,t}$ and $\mathbb{I}_{\text{eng}}^{n,t} r_{\text{guide}}^{n,t}$, and $r_{\text{col}}^{n,t} + r_{\text{con}}^{n,t} + r_{\text{win}}^{n,t} + r_{\text{time}}^{n,t}$.

D. Training Strategy via Probabilistic Prior Injection

While DSGN is solver-agnostic, we employ IQN to model distributional returns. Optimizing the vector gate $g_n^t \in (0,1)^d$ solely via extrinsic rewards can yield ambiguous weights, e.g., $g_n^t \approx 0.5\mathbf{1}_d$, where $\mathbf{1}_d$ is the d-dimensional all-one vector. We therefore regularize each channel $g_{n,j}^t$, $j = 1,\dots,d$, with a state-dependent Beta prior whose parameters depend on M_{vis}^t: $(\alpha,\beta) = (1,4)$ if $M_{\text{vis}}^t = 0$, and $(\alpha,\beta) = (0.5, 0.5)$ if $M_{\text{vis}}^t = 1$. The resulting negative log-likelihood regularizer $\mathcal{L}_{\text{reg}}(g_n^t, M_{\text{vis}}^t)$ is the sum over $j = 1,\dots,d$ of $(\alpha(M_{\text{vis}}^t) - 1)\ln g_{n,j}^t$ and $(\beta(M_{\text{vis}}^t) - 1)\ln(1 - g_{n,j}^t)$, multiplied by $-1/d$.

Fig. 3 illustrates the state-dependent priors applied to each gate channel. When no evader is observed, $(\alpha,\beta) = (1,4)$ drives $g_{n,j}^t \to 0$ and suppresses noisy containment activation. When evaders are observed, $\alpha = \beta = 0.5$ penalizes ambiguous gates near 0.5 and encourages decisive arbitration between surveillance and containment.

To prevent the prior from dominating early representation learning, we linearly increase its coefficient λ_t after a warm-up phase. The final objective $\mathcal{L}_{\text{total}}$ sums the average quan-

TABLE I
PROGRESSIVE CURRICULUM SCHEDULE

Training Steps (10^6)	Pursuers	Evaders	Obstacles
$0 \le T_{\text{train}} < 2$	3	1	0
$2 \le T_{\text{train}} < 4$	4	1	1
$4 \le T_{\text{train}} < 5$	7	2	2
$5 \le T_{\text{train}} < 6$	11	3	4
$6 \le T_{\text{train}} \le 7$	15	4	6

tile Huber loss $N_q^{-1} \sum_{q=1}^{N_q} \rho_{\tau_q}^{\kappa}(\delta_{\tau_q})$ [28] and the weighted gate regularizer $\lambda_t \mathcal{L}_{\text{reg}}(g_n^t, M_{\text{vis}}^t)$, where δ_{τ_q} is the temporal-difference error at sampled quantile $\tau_q \sim U([0,1])$, N_q is the number of quantile samples, and κ is the Huber threshold.

IV. EXPERIMENTS

A. Training Setup

We extend the open-source multi-robot navigation simulator described in [30] by retaining its kinematic, sensing, and collision models and adding surveillance nodes, reactive evaders, and multi-target containment. The workspace is discretized into a uniform rectangular grid, and one surveillance node c_k is placed at the center of each grid cell for persistent surveillance and idleness computation. For an environment of size $W \times H$, the maximum grid-cell side length is $L_{\max} = \eta R_{\text{sense}} \sqrt{2}$, ensuring that half the diagonal of each cell is no larger than ηR_{sense}. With $R_{\text{sense}} = 20\,\text{m}$ and $\eta = 0.95$, this gives $L_{\max} \approx 26.9\,\text{m}$. The numbers of grid cells along the two axes are $N_x = \lceil W/L_{\max} \rceil$ and $N_y = \lceil H/L_{\max} \rceil$, with cell dimensions $\Delta_x = W/N_x$ and $\Delta_y = H/N_y$.

During training, each agent's input includes at most the nearest 5 obstacles, 5 teammates, 8 evaders, and 16 surveillance nodes; unused entries are zero-padded. Successful containment follows Eq. (1) with $\lambda_{\text{uni}} = 3$. A training episode terminates when all evaders are contained, fewer than three pursuers remain active, or the episode exceeds 3000 time steps. To ease learning in the early stages, training follows a progressive curriculum that gradually increases the number of entities, as shown in Table I.

B. Evaluation Setup

Without retraining, each policy is tested across nine scale–density configurations (Table II), ranging from 12v3 (12 pursuers versus 3 evaders) to 77v19. The largest evaluation setting, 77v19, exceeds the largest curriculum training setting, 15v4. Within each density level, the environment size varies with team size to maintain approximately constant mobile-entity density across map scales. This design helps separate the effects of increasing team and map sizes from changes in overall crowding. All experiments use a consistent rectangular outer boundary. Except for the separately reported action-granularity test in Section IV-E4, each method and arbitration variant is evaluated over 40 trials per random seed, and across 5 independent random seeds in every task scenario. Initial poses of pursuers, evaders, and obstacles are randomized in each trial. For these comparisons, results are reported as seed-level means across seeds with 95% confidence intervals. These tests assess structural generalization under shared simulator dynamics. The evaluation horizon is 1000 time steps, corresponding to 500s of simulated time.

TABLE II
CONFIGURATIONS FOR CROSS-SCALE EVALUATION

Map Scale	Region Size (m×m)	Pursuers	Evaders	Obstacles	Density Level ρ_m (10^{-3} m^{-2})
Small	112×112	12	3	3	Low (1.196)
		16	4	3	Medium (1.594)
		20	5	3	High (1.993)
Medium	200×200	38	10	10	Low (1.200)
		51	13	10	Medium (1.600)
		64	16	10	High (2.000)
Large	220×220	46	12	12	Low (1.198)
		62	15	12	Medium (1.591)
		77	19	12	High (1.983)

Note: $\rho_m = (N+M)/(WH)$ denotes the mobile-entity density.

We impose kinematic asymmetry so that successful containment requires cooperation. Following the discrete kinematic command setting in [30], pursuers are bounded by $v_{\max} = 3.0$ m/s and use a 3×3 action space ($a_{\text{acc}} \in \{0, \pm 0.4\}$ m/s^2, $\omega \in \{0, \pm\pi/6\}$ rad/s). These actions form a platform-agnostic high-level command interface; related learned pursuit policies have been deployed in real-world settings [11]. Evaders are guided by a reactive artificial potential field (APF) [31] and use $v_{\max} = 3.5$ m/s with a finer rotational spectrum ($\omega \in \{0, \pm\pi/12, \pm\pi/6\}$ rad/s) under the same acceleration limits. This kinematic difference precludes naive individual pursuit.

We evaluate cross-scale performance using five metrics. *Strict Success Rate* (SR) is the fraction of trials in which all evaders are contained without collisions. *Containment Time* (T_{cap}) measures the time from episode start to containment completion, conditioned on episodes that complete containment. *Collision Ratio* (CR) is computed per episode as the number of pursuers that experience at least one collision with an obstacle or teammate, divided by the total number of pursuers; repeated collisions by the same pursuer are counted once. *Containment Angle Variance* (Var) is computed when an individual evader is contained and measures the angular uniformity of the containment formation. *Patrol Completion Steps* (P-Steps) measures the steps from containment completion to final surveillance coverage, conditioned on completion of both stages.

C. Baseline Methods

We select representative baselines relevant to decentralized decision-making, multi-agent motion planning, and reinforcement learning, and adapt them to the SSC setting. We compare DSGN with TransIQN [6], IQN [28], Deep Q-Network (DQN) [23], and a reactive Heuristic baseline based on APF [31] under identical settings. For the main comparisons, all learned methods use the same shaped reward (Section III-C), progressive curriculum (Table I), action space, and 7M steps training budget per seed. The TransIQN baseline uses the same transformer backbone as DSGN but represents surveillance and containment in a single ego-token feature, providing a direct comparison between shared and task-specific relational representations. To control for model

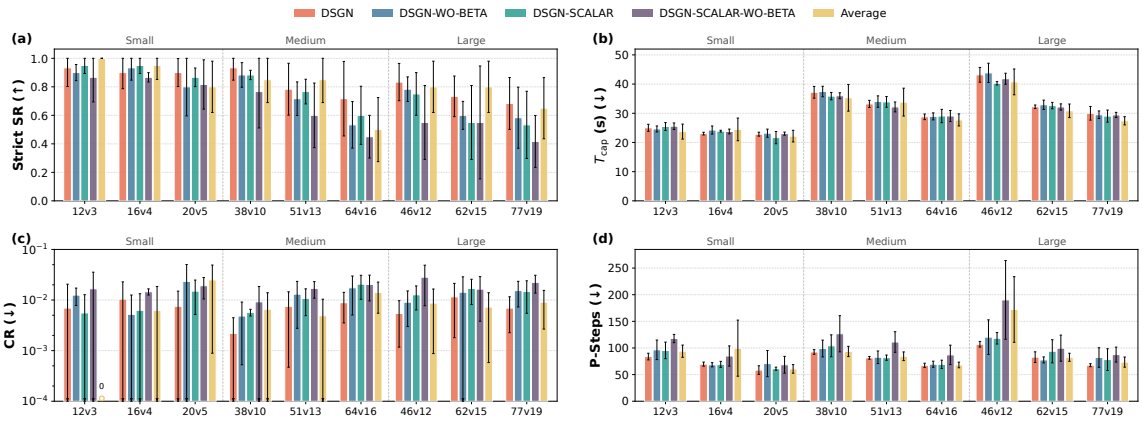


Fig. 4. **Arbitration Variants across Nine Scale–Density Scenarios. (a)** Strict SR; **(b)** T_{cap} (s); **(c)** CR (log scale); **(d)** P-Steps. Error bars show 95% confidence intervals across 5 independent training seeds, with 40 trials per seed and scenario.

capacity, its parameter count is matched to that of DSGN using a residual expansion block. IQN and DQN serve as representative value-based learning baselines, but both compress environmental features through average pooling and therefore do not perform explicit relational reasoning. The Heuristic baseline uses manually designed attractive and repulsive fields as a reactive policy without learning.

D. Results and Discussion

Fig. 4 compares DSGN, its three gate and prior ablations, and the fixed equal-weight Average variant across the nine configurations in Table II; Table A.I summarizes the results by scale.

Task Completion: DSGN has the highest or tied-highest scale-averaged Strict SR at all three scales in Fig. 4(a), despite not leading in every configuration. This supports more reliable task completion through separate surveillance and containment streams.

Containment Efficiency: In Fig. 4(b), DSGN's scale-averaged T_{cap} exceeds the fastest variant by only 0.1–2.1 s. Its short times suggest that relational attention aids spatial assignment around evaders, while dual-stream processing adds little execution delay.

Safety: DSGN has the lowest scale-averaged CR at all three scales in Fig. 4(c), although the best variant varies by configuration. This suggests that disentangled task processing reduces interference between surveillance and containment behaviors.

Post-Containment Surveillance: DSGN has the lowest scale-averaged P-Steps at all three scales in Fig. 4(d), despite some configuration-level exceptions. This suggests that its surveillance stream supports prompt coverage completion after containment.

Together, Fig. 4 and Table III suggest that task-specific streams complement relational reasoning: unlike a shared stream that may mix surveillance and containment cues, DSGN addresses this issue by separating the two objectives into specialized streams and coordinating them through context-aware gating. This design supports safer and more consistent multi-robot coordination as the number of interacting agents increases.

E. Ablation Study

To analyze task arbitration and mode switching, we conduct structural ablations and phase-specific evaluations (Tables III–V and Table A.I); we also assess sensitivity to search-region geometry and action granularity (Tables A.II and A.III). Random replaces the learned gate with a random gate; Average assigns equal weights to the two streams; Rule-based uses a hand-crafted spatial rule for arbitration; DSGN-WO-BETA removes the Beta-prior regularization; DSGN-SCALAR replaces vector-wise gating with a single scalar gate; and DSGN-SCALAR-WO-BETA combines scalar gating without the Beta prior.

1) Dynamic Gating and Probabilistic Priors: Due to space constraints, Table IV presents only the Large-scale results. The complete comparison across Small, Medium, and Large scales is provided in Table A.I in Appendix A; each scale aggregates the Low, Medium, and High density configurations in Table II. Removing either the Beta prior or vector-wise gating worsens scale-averaged SR, CR, Var, and P-Steps at all three scales (Table A.I). This supports both components' roles in reliable task arbitration, while Average retains a modest T_{cap} advantage.

2) Mode Switching Across Task Phases: SSC requires reliable switching between surveillance and containment. We evaluate this capability in two simulation settings: Pure Surveillance and Task Switching (Table III). Pure Surveillance contains no evaders and requires the team to visit all surveillance nodes within the evaluation horizon. Task Switching starts with the same surveillance objective, and evaders are later introduced near the boundary of the active pursuers' sensing range. Success in this setting requires both complete surveillance-node coverage and successful evader containment, thereby testing the transition from global dispersion to tactical containment.

Table III shows that DSGN has the shortest mean Task Switching episode duration in all settings and Pure Surveillance SR of 1.000 at every tested team size. Random's faster

TABLE III
MODE SWITCHING AND PURE SURVEILLANCE PERFORMANCE. EACH ENTRY REPORTS SR / TIME (S), WITH 95% CONFIDENCE INTERVALS IN BRACKETS.

(a) Task Switching (SR ↑ /Time ↓)

Method	**3v1**	**6v2**	**9v3**
Heuristic	0.000 / 488.4 [0.000, 0.000] / [465.0, 511.9]	0.000 / 500.0 [0.000, 0.000] / [500.0, 500.0]	0.000 / 500.0 [0.000, 0.000] / [500.0, 500.0]
DQN	0.000 / 500.0 [0.000, 0.000] / [500.0, 500.0]	0.000 / 500.0 [0.000, 0.000] / [500.0, 500.0]	0.000 / 500.0 [0.000, 0.000] / [500.0, 500.0]
IQN	0.000 / 451.3 [0.000, 0.000] / [415.4, 487.1]	0.950 / 169.5 [0.879, 1.000] / [155.4, 183.6]	0.785 / 112.5 [0.650, 0.920] / [106.7, 118.3]
TransIQN	0.025 / 455.0 [0.000, 0.076] / [413.8, 496.2]	0.840 / 190.8 [0.724, 0.956] / [160.6, 220.9]	0.725 / 103.9 [0.580, 0.870] / [84.8, 122.9]
Random	0.000 / 467.2 [0.000, 0.000] / [429.6, 504.8]	0.075 / 475.8 [0.000, 0.160] / [450.3, 501.4]	0.300 / 405.7 [0.152, 0.448] / [359.3, 452.1]
Average	0.025 / 487.7 [0.000, 0.076] / [463.9, 511.5]	0.665 / 337.7 [0.513, 0.817] / [289.3, 386.0]	0.820 / 146.6 [0.690, 0.950] / [106.6, 186.6]
Rule-based	**1.000** / 175.5 [1.000, 1.000] / [164.3, 186.7]	0.925 / 109.4 [0.840, 1.000] / [102.8, 115.9]	0.650 / 91.3 [0.496, 0.804] / [85.1, 97.5]
DSGN-WO-BETA	0.165 / 473.7 [0.042, 0.288] / [451.8, 495.5]	**0.975** / 128.1 [0.924, 1.000] / [118.7, 137.4]	0.875 / 86.1 [0.768, 0.982] / [81.3, 90.8]
DSGN-SCALAR	0.975 / 225.1 [0.924, 1.000] / [204.0, 246.1]	**0.975** / 122.7 [0.924, 1.000] / [99.4, 146.1]	0.850 / 84.6 [0.734, 0.966] / [78.0, 91.2]
DSGN-SCALAR-WO-BETA	0.925 / 317.6 [0.840, 1.000] / [287.2, 347.9]	0.925 / 138.6 [0.840, 1.000] / [125.2, 152.0]	0.810 / 93.9 [0.680, 0.940] / [82.1, 105.7]
DSGN	**1.000 / 162.2** [1.000, 1.000] / [150.6, 173.7]	0.950 / **98.0** [0.879, 1.000] / [93.0, 103.0]	**0.925 / 76.7** [0.840, 1.000] / [72.2, 81.3]

(b) Pure Surveillance, where N denotes the number of pursuers (SR ↑ /Time ↓)

Method	$N = 3$	$N = 4$	$N = 5$
Heuristic	0.000 / 500.0 [0.000, 0.000] / [500.0, 500.0]	0.000 / 500.0 [0.000, 0.000] / [500.0, 500.0]	0.000 / 500.0 [0.000, 0.000] / [500.0, 500.0]
DQN	0.000 / 500.0 [0.000, 0.000] / [500.0, 500.0]	0.000 / 500.0 [0.000, 0.000] / [500.0, 500.0]	0.075 / 494.1 [0.000, 0.160] / [486.9, 501.2]
IQN	0.000 / 444.6 [0.000, 0.000] / [410.2, 479.0]	0.385 / 432.9 [0.228, 0.542] / [400.6, 465.2]	0.900 / 288.5 [0.803, 0.997] / [253.6, 323.4]
TransIQN	0.175 / 436.1 [0.052, 0.298] / [400.3, 472.0]	0.715 / 294.2 [0.570, 0.860] / [244.1, 344.4]	**1.000** / 131.7 [1.000, 1.000] / [112.4, 151.1]
Random	**1.000** / 130.7 [1.000, 1.000] / [122.0, 139.3]	0.975 / 104.0 [0.924, 1.000] / [97.0, 110.9]	**1.000 / 82.5** [1.000, 1.000] / [77.0, 88.0]
Average	0.250 / 455.3 [0.110, 0.390] / [425.6, 485.0]	0.620 / 385.8 [0.451, 0.769] / [344.1, 427.5]	0.875 / 245.9 [0.768, 0.982] / [204.6, 287.1]
Rule-based	**1.000** / 137.7 [1.000, 1.000] / [129.0, 146.3]	**1.000** / 105.2 [1.000, 1.000] / [99.1, 111.2]	**1.000** / 84.8 [1.000, 1.000] / [80.5, 89.0]
DSGN-WO-BETA	0.660 / 379.5 [0.506, 0.814] / [340.8, 418.1]	0.975 / 200.7 [0.924, 1.000] / [170.4, 231.1]	**1.000** / 108.9 [1.000, 1.000] / [99.4, 118.5]
DSGN-SCALAR	0.975 / 177.4 [0.924, 1.000] / [166.4, 188.5]	**1.000** / 167.8 [1.000, 1.000] / [148.5, 187.0]	**1.000** / 110.8 [1.000, 1.000] / [98.4, 123.2]
DSGN-SCALAR-WO-BETA	0.925 / 287.8 [0.840, 1.000] / [252.1, 323.4]	0.950 / 137.2 [0.879, 1.000] / [113.4, 161.0]	0.975 / 102.9 [0.924, 1.000] / [81.8, 123.9]
DSGN	**1.000 / 124.6** [1.000, 1.000] / [118.1, 131.1]	**1.000 / 100.3** [1.000, 1.000] / [95.6, 105.1]	**1.000** / 83.3 [1.000, 1.000] / [77.9, 88.7]

Note: Task-switching success requires complete coverage and containment. Time is mean episode duration over all trials, capped at 500 s per trial; confidence intervals are not clipped.

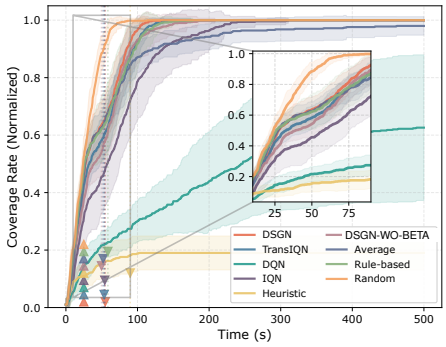


Fig. 5. Temporal coverage dynamics in the 6v2 task-switching scenario.

$N = 5$ patrol does not translate to reliable switching. Under scalar gating, the Beta prior improves Task Switching SR and shortens mean episode duration in every configuration, while raising Pure Surveillance SR at every tested team size.

To further clarify the switching process, we analyze the temporal coverage curves using two quantities: the normalized coverage at the switching instant, denoted by C_{switch}, and the post-containment surveillance recovery time, denoted by T_{resume}, which measures the time from containment completion to final coverage completion. T_{resume} is computed only for episodes that complete both containment and final coverage; incomplete episodes are excluded rather than assigned the horizon value, while SR is computed over all episodes. Because coverage is cumulative, Fig. 5 reflects surveillance interruption and recovery rather than containment success alone. The upward triangles mark evader spawning at $t = 25$ s, while the downward triangles and dotted lines mark average containment completion. The coverage trajectory typically follows an "increase–flatten–increase" pattern: surveillance progress accumulates before the switching event, slows down during containment, and resumes after all evaders are contained. Table V summarizes these temporal indicators. DSGN has the highest mean C_{switch} and shortest mean T_{resume} in all three settings (Table V). Its advantage is more pronounced in recovery time, indicating that DSGN chiefly improves how quickly surveillance resumes after containment.

TABLE IV
LARGE-SCALE ARBITRATION VARIANTS (46V12–77V19). EACH CELL REPORTS THE MEAN WITH A 95% CONFIDENCE INTERVAL IN BRACKETS.

Variant	SR ↑	T_{cap} (s) ↓	CR ↓	Var ↓	P-Steps ↓
Random	0.000 [0.000, 0.000]	380.9 [303.6, 458.1]	0.1403 [0.1173, 0.1634]	2.392 [2.276, 2.507]	124.8 [113.8, 135.8]
Average	**0.750** [0.640, 0.860]	**33.0** [30.8, 35.2]	0.0083 [0.0044, 0.0123]	0.836 [0.752, 0.919]	109.4 [85.9, 133.0]
Rule-based	0.400 [0.275, 0.525]	47.6 [43.5, 51.6]	0.0320 [0.0219, 0.0422]	1.056 [0.972, 1.140]	113.6 [105.2, 122.0]
DSGN-WO-BETA	0.656 [0.540, 0.771]	35.4 [33.4, 37.5]	0.0129 [0.0044, 0.0214]	0.756 [0.617, 0.894]	93.3 [74.7, 112.0]
DSGN-SCALAR	0.611 [0.425, 0.797]	34.0 [32.8, 35.2]	0.0148 [0.0074, 0.0222]	0.792 [0.777, 0.807]	96.7 [79.3, 114.2]
DSGN-SCALAR-WO-BETA	0.506 [0.241, 0.770]	34.5 [33.7, 35.2]	0.0223 [0.0095, 0.0351]	0.810 [0.715, 0.905]	125.8 [98.0, 153.7]
DSGN	**0.750** [0.603, 0.897]	35.1 [33.7, 36.6]	**0.0080** [0.0018, 0.0141]	**0.751** [0.657, 0.845]	**85.8** [82.6, 89.1]

3) Sensitivity to Search-Region Geometry: We evaluate 6v2 task switching under three search-region layouts: G0 has no holes, G2 has one central hexagonal hole, and G3 has four triangular holes (Fig. A.1 in Appendix A). In both G2 and G3, the holes occupy 20.0% of the workspace; the outer boundary is unchanged. Comparing these equal-area hole layouts tests sensitivity to spatial arrangement rather than blocked fraction. DSGN has the highest mean SR and lowest mean P-Steps in each layout (Table A.II), indicating the strongest combined task completion and post-containment surveillance despite not always leading in time or CR. Its SR still drops from 1.000 in G0 to 0.500 in G3, showing that complex layouts remain challenging despite its relative advantage.

4) Sensitivity to Action Granularity: We compare separately trained DSGN policies with 3×3 and 5×5 command grids in 3v1, 6v2, and 9v3 Task Switching (Table A.III). The finer grid adds intermediate commands within the same bounds. The 5×5 grid has mean T_{cap} and P-Steps close to those of 3×3, with slightly shorter containment times in 6v2 and 9v3 and fewer P-Steps in 3v1 and 6v2. Although its SR is lower and CR higher in all three settings, DSGN remains effective with the finer action grid, demonstrating good adaptability to higher-resolution action spaces while

TABLE V
TEMPORAL INDICATORS FOR MODE SWITCHING

Scenario	C_{switch} ↑					T_{resume} (s) ↓				
	DSGN	DSGN-WO-BETA	DSGN-SCALAR	DSGN-SCALAR-WO-BETA	TransIQN	DSGN	DSGN-WO-BETA	DSGN-SCALAR	DSGN-SCALAR-WO-BETA	TransIQN
3v1	**0.252**	0.232	0.234	0.240	0.237	**176.4**	212.4	250.5	269.5	268.5
	[0.216, 0.288]	[0.169, 0.294]	[0.217, 0.251]	[0.205, 0.275]	[0.201, 0.273]	**[83.1, 269.7]**	[116.3, 308.5]	[198.8, 302.2]	[28.0, 511.1]	[153.8, 383.1]
6v2	**0.466**	0.436	0.440	0.455	0.446	**55.5**	69.3	89.0	100.6	99.0
	[0.388, 0.544]	[0.329, 0.543]	[0.405, 0.475]	[0.395, 0.515]	[0.409, 0.483]	**[35.4, 75.7]**	[39.5, 99.0]	[65.3, 112.7]	[53.8, 147.5]	[26.9, 171.1]
9v3	**0.636**	0.581	0.610	0.620	0.608	**31.6**	36.3	41.4	45.9	56.9
	[0.551, 0.721]	[0.470, 0.691]	[0.590, 0.630]	[0.566, 0.673]	[0.563, 0.654]	**[20.9, 42.4]**	[23.9, 48.8]	[36.3, 46.5]	[33.1, 58.8]	[17.4, 96.5]

Note: C_{switch} denotes coverage at switching; T_{resume} denotes recovery time in seconds. Brackets denote 95% confidence intervals. T_{resume} is conditioned on episodes that complete both containment and final coverage.

maintaining comparable execution efficiency.

V. CONCLUSION

DSGN separates surveillance and containment representations and coordinates them through vector-wise gating with a state-dependent Beta prior. Experiments show the benefits of DSGN for task completion and post-containment surveillance recovery at different team sizes. Future work will investigate its application in continuous action spaces, repeated target-appearance cycles, and three-dimensional nonholonomic robots with limited communication.

REFERENCES

[1] G. Yang *et al.*, "The grand challenges of science robotics," *Sci. Robot.*, vol. 3, no. 14, Jan. 2018, Art. no. eaar7650, doi: 10.1126/scirobotics.aar7650.

[2] M. Dorigo, G. Theraulaz, and V. Trianni, "Swarm robotics: Past, present, and future [point of view]," *Proc. IEEE*, vol. 109, no. 7, pp. 1152–1165, Jul. 2021, doi: 10.1109/JPROC.2021.3072740.

[3] L. Vanhée, M. Borit, and J. M. Santos, "Autonomous fishing vessels roving the seas: What multiagent systems have got to do with it," in *Proc. 17th Int. Conf. Auton. Agents Multiagent Syst. (AAMAS)*, Jul. 2018, pp. 1193–1197.

[4] D. Liu, W. Bao, X. Zhu, B. Fei, T. Men, and Z. Xiao, "Cooperative path optimization for multiple UAVs surveillance in uncertain environment," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 10 676–10 692, Jul. 2022, doi: 10.1109/JIOT.2021.3125784.

[5] Y. Wang, Y. Wang, Y. Cao, and G. Sartoretti, "Spatio-temporal attention network for persistent monitoring of multiple mobile targets," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2023, pp. 3903–3910.

[6] H. Zhang, G. Zhao, and X. Ren, "TERL: Large-scale multi-target encirclement using transformer-enhanced reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2025, pp. 13 438–13 445, doi: 10.1109/IROS60139.2025.11246832.

[7] G. Qu, Y. Lin, A. Wierman, and N. Li, "Scalable multi-agent reinforcement learning for networked systems with average reward," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 2074–2086.

[8] L. Buşoniu, R. Babuška, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 38, no. 2, pp. 156–172, Mar. 2008, doi: 10.1109/TSMCC.2007.913919.

[9] I. E. Weintraub, M. Pachter, and E. Garcia, "An introduction to pursuit-evasion differential games," in *Proc. Amer. Control Conf. (ACC)*, 2020, pp. 1049–1066.

[10] J. Wang, G. Li, L. Liang, C. Wang, and F. Deng, "Pursuit-evasion games of multiple cooperative pursuers and an evader: A biological-inspired perspective," *Commun. Nonlinear Sci. Numer. Simul.*, vol. 110, Jul. 2022, Art. no. 106386, doi: 10.1016/j.cnsns.2022.106386.

[11] Z. Zhang, X. Wang, Q. Zhang, and T. Hu, "Multi-robot cooperative pursuit via potential field-enhanced reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2022, pp. 8808–8814.

[12] Y. Wang, L. Dong, and C. Sun, "Cooperative control for multi-player pursuit-evasion games with reinforcement learning," *Neurocomputing*, vol. 412, pp. 101–114, Oct. 2020.

[13] H. Yang, P. Ge, J. Cao, Y. Yang, and Y. Liu, "Large scale pursuit-evasion under collision avoidance using deep reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2023, pp. 2232–2239.

[14] A. Bonato, P. A. Golovach, G. Hahn, and J. Kratochvíl, "The capture time of a graph," *Discrete Math.*, vol. 309, no. 18, pp. 5588–5595, Sep. 2009, doi: 10.1016/j.disc.2008.04.004.

[15] L. J. Guibas, J.-C. Latombe, S. M. LaValle, D. Lin, and R. Motwani, "A visibility-based pursuit-evasion problem," *Int. J. Comput. Geom. Appl.*, vol. 9, no. 4–5, pp. 471–493, Aug. 1999, doi: 10.1142/S0218195999000273.

[16] S.-M. Park, J.-H. Lee, and K.-Y. Chwa, "Visibility-based pursuit-evasion in a polygonal region by a searcher," in *Proc. Int. Colloq. Automata, Lang., Program. (ICALP)*, ser. Lecture Notes in Computer Science, vol. 2076. Springer, 2001, pp. 456–468, doi: 10.1007/3-540-48224-5_38.

[17] S. M. LaValle and J. E. Hinrichsen, "Visibility-based pursuit-evasion: The case of curved environments," *IEEE Trans. Robot. Autom.*, vol. 17, no. 2, pp. 196–202, Apr. 2001, doi: 10.1109/70.928565.

[18] G. A. Hollinger, S. Singh, J. Djugash, and A. Kehagias, "Efficient multi-robot search for a moving target," *Int. J. Robot. Res.*, vol. 28, no. 2, pp. 201–219, Feb. 2009, doi: 10.1177/0278364908099853.

[19] S. Agrawal, S. Bhore, J. S. B. Mitchell, P. B. Sujit, and A. Gohil, "Multi-robot searching with limited sensing range for static and mobile intruders," in *Proc. IEEE Int. Conf. Emerg. Technol. Auton. Aerial Vehicles (ETAAV)*, 2025, pp. 1–7, doi: 10.1109/ETAAV66793.2025.11213328.

[20] S. Ye, Y. Li, and H. Liu, "Deep reinforcement learning based multi-robot search for mobile targets," in *Proc. 21st Int. Conf. Mobility, Sens. Netw. (MSN)*, 2025, pp. 30–37, doi: 10.1109/MSN69125.2025.00013.

[21] D. Liberzon, *Switching in Systems and Control*. Boston, MA, USA: Birkhäuser, 2003. doi: 10.1007/978-1-4612-0017-8.

[22] M. S. Branicky, "Multiple Lyapunov functions and other analysis tools for switched and hybrid systems," *IEEE Trans. Autom. Control*, vol. 43, no. 4, pp. 475–482, Apr. 1998.

[23] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015, doi: 10.1038/nature14236.

[24] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn, "Gradient surgery for multi-task learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 5824–5836.

[25] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Comput.*, vol. 3, no. 1, pp. 79–87, Feb. 1991, doi: 10.1162/neco.1991.3.1.79.

[26] J. Andreas, D. Klein, and S. Levine, "Modular multitask reinforcement learning with policy sketches," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, vol. 70, Aug. 2017, pp. 166–175.

[27] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs*, ser. SpringerBriefs in Intelligent Systems. Cham, Switzerland: Springer, 2016. doi: 10.1007/978-3-319-28929-8.

[28] W. Dabney, G. Ostrovski, D. Silver, and R. Munos, "Implicit quantile networks for distributional reinforcement learning," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, vol. 80, Jul. 2018, pp. 1096–1105.

[29] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. 16th Int. Conf. Mach. Learn. (ICML)*, 1999, pp. 278–287.

[30] X. Lin, Y. Huang, F. Chen, and B. Englot, "Decentralized multi-robot navigation for autonomous surface vehicles with distributional reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 8327–8333.

[31] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robot. Res.*, vol. 5, no. 1, pp. 90–98, Mar. 1986.

APPENDIX A
ADDITIONAL EXPERIMENTAL RESULTS

TABLE A.I
COMPARISON OF ARBITRATION VARIANTS ACROSS SMALL, MEDIUM, AND LARGE SCALES

(a) Small scale: 12v3–20v5

Variant	SR ↑	T_{cap} (s) ↓	CR ↓	Var ↓	P-Steps ↓
Random	0.433 [0.253, 0.614]	128.4 [83.4, 173.4]	0.0543 [0.0317, 0.0770]	2.267 [2.050, 2.484]	87.9 [77.0, 98.8]
Average	0.917 [0.846, 0.987]	**23.5** [21.8, 25.1]	0.0104 [0.0012, 0.0197]	0.839 [0.707, 0.970]	84.8 [66.4, 103.1]
Rule-based	0.800 [0.698, 0.902]	32.4 [30.2, 34.6]	0.0245 [0.0108, 0.0383]	1.290 [1.155, 1.425]	88.0 [81.4, 94.7]
DSGN-WO-BETA	0.878 [0.771, 0.985]	24.0 [22.8, 25.3]	0.0137 [0.0016, 0.0258]	0.848 [0.786, 0.910]	78.5 [74.7, 82.4]
DSGN-SCALAR	0.911 [0.826, 0.996]	23.7 [22.8, 24.6]	0.0089 [0.0011, 0.0168]	0.870 [0.777, 0.963]	75.3 [67.2, 83.3]
DSGN-SCALAR-WO-BETA	0.850 [0.726, 0.974]	24.1 [23.8, 24.4]	0.0168 [0.0071, 0.0265]	0.830 [0.786, 0.875]	90.4 [77.4, 103.3]
DSGN	**0.922** [0.865, 0.980]	23.6 [23.3, 23.9]	**0.0083** [0.0007, 0.0159]	**0.789** [0.722, 0.856]	**70.7** [64.6, 76.7]

(b) Medium scale: 38v10–64v16

Variant	SR ↑	T_{cap} (s) ↓	CR ↓	Var ↓	P-Steps ↓
Random	0.000 [0.000, 0.000]	356.4 [261.5, 451.3]	0.1426 [0.1142, 0.1709]	2.442 [2.299, 2.585]	118.7 [103.8, 133.7]
Average	0.733 [0.620, 0.846]	**32.3** [29.9, 34.7]	0.0085 [0.0043, 0.0128]	0.770 [0.683, 0.858]	82.0 [76.8, 87.2]
Rule-based	0.517 [0.389, 0.644]	44.5 [39.8, 49.2]	0.0298 [0.0140, 0.0455]	1.042 [0.944, 1.140]	105.4 [96.3, 114.6]
DSGN-WO-BETA	0.711 [0.607, 0.815]	33.5 [31.9, 35.0]	0.0118 [0.0055, 0.0181]	0.741 [0.624, 0.859]	83.7 [72.8, 94.5]
DSGN-SCALAR	0.750 [0.668, 0.832]	32.9 [31.7, 34.2]	0.0124 [0.0101, 0.0146]	0.765 [0.652, 0.878]	85.0 [74.3, 95.8]
DSGN-SCALAR-WO-BETA	0.606 [0.397, 0.814]	32.4 [32.1, 32.7]	0.0155 [0.0072, 0.0238]	0.773 [0.728, 0.818]	108.3 [88.5, 128.1]
DSGN	**0.811** [0.637, 0.985]	33.1 [31.8, 34.4]	**0.0062** [0.0016, 0.0108]	**0.727** [0.710, 0.744]	**80.5** [79.1, 81.8]

(c) Large scale: 46v12–77v19

Variant	SR ↑	T_{cap} (s) ↓	CR ↓	Var ↓	P-Steps ↓
Random	0.000 [0.000, 0.000]	380.9 [303.6, 458.1]	0.1403 [0.1173, 0.1634]	2.392 [2.276, 2.507]	124.8 [113.8, 135.8]
Average	**0.750** [0.640, 0.860]	**33.0** [30.8, 35.2]	0.0083 [0.0044, 0.0123]	0.836 [0.752, 0.919]	109.4 [85.9, 133.0]
Rule-based	0.400 [0.275, 0.525]	47.6 [43.5, 51.6]	0.0320 [0.0219, 0.0422]	1.056 [0.972, 1.140]	113.6 [105.2, 122.0]
DSGN-WO-BETA	0.656 [0.540, 0.771]	35.4 [33.4, 37.5]	0.0129 [0.0044, 0.0214]	0.756 [0.617, 0.894]	93.3 [74.7, 112.0]
DSGN-SCALAR	0.611 [0.425, 0.797]	34.0 [32.8, 35.2]	0.0148 [0.0074, 0.0222]	0.792 [0.777, 0.807]	96.7 [79.3, 114.2]
DSGN-SCALAR-WO-BETA	0.506 [0.241, 0.770]	34.5 [33.7, 35.2]	0.0223 [0.0095, 0.0351]	0.810 [0.715, 0.905]	125.8 [98.0, 153.7]
DSGN	**0.750** [0.603, 0.897]	35.1 [33.7, 36.6]	**0.0080** [0.0018, 0.0141]	**0.751** [0.657, 0.845]	**85.8** [82.6, 89.1]

Note: Brackets denote 95% confidence intervals. T_{cap} is conditioned on containment completion; P-Steps is conditioned on successful containment and final coverage.

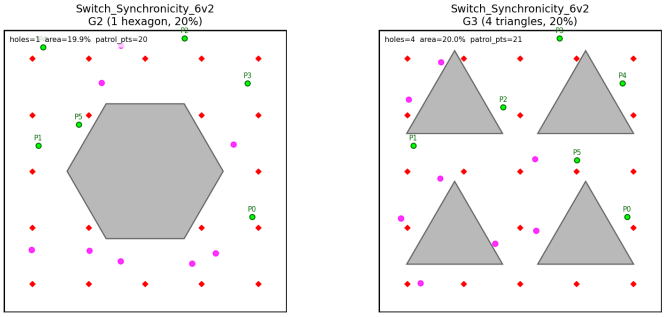


Fig. A.1. Representative 6v2 layouts: G2 (one hexagonal hole, left) and G3 (four triangular holes, right). Green circles labeled P0–P5 are pursuers, magenta circles are obstacles, and red diamonds are surveillance nodes. Gray areas are inaccessible. G0 has no hole.

TABLE A.II
6V2 GEOMETRY SENSITIVITY ACROSS DSGN VARIANTS AND TRANSIQN

(a) G0: no hole

Method	SR↑	$\mathbf{T}_{\mathrm{cap}}$ (s)↓	CR↓	P-Steps↓
TransIQN	0.950 [0.765,0.990]	**26.1** [20.8,31.4]	0.0083 [0.0000,0.0258]	227.8 [168.9,286.7]
D-w/o-B	0.935 [0.790,1.000]	30.3 [25.0,35.6]	0.0083 [0.0000,0.0290]	129.6 [109.4,149.8]
D-S	0.965 [0.825,1.000]	28.7 [20.6,36.8]	0.0111 [0.0000,0.0589]	186.3 [86.2,286.3]
D-S-w/o-B	0.885 [0.485,1.000]	30.0 [23.2,36.8]	0.0083 [0.0000,0.0290]	229.3 [186.8,271.7]
DSGN	**1.000** [1.000,1.000]	27.1 [18.4,35.8]	**0.0000** [0.0000,0.0000]	**89.7** [40.5,139.0]

(b) G2: one hexagonal hole

Method	SR↑	$\mathbf{T}_{\mathrm{cap}}$ (s)↓	CR↓	P-Steps↓
TransIQN	0.650 [0.435,0.820]	**42.0** [29.3,54.7]	0.0833 [0.0188,0.1479]	244.6 [163.9,325.3]
D-w/o-B	0.835 [0.435,1.000]	55.0 [52.6,57.4]	0.0444 [0.0000,0.1640]	107.5 [85.1,129.8]
D-S	0.815 [0.560,1.000]	44.6 [36.0,53.1]	0.0583 [0.0000,0.1532]	144.8 [137.1,152.4]
D-S-w/o-B	0.765 [0.510,1.000]	52.4 [10.2,94.6]	0.0556 [0.0239,0.0872]	177.9 [146.4,209.4]
DSGN	**0.865** [0.675,1.000]	43.3 [27.4,59.2]	**0.0361** [0.0242,0.0481]	**78.5** [25.2,131.8]

(c) G3: four triangular holes

Method	SR↑	$\mathbf{T}_{\mathrm{cap}}$ (s)↓	CR↓	P-Steps↓
TransIQN	0.350 [0.180,0.565]	**61.5** [34.8,88.2]	0.1583 [0.0689,0.2477]	388.0 [299.6,476.4]
D-w/o-B	0.485 [0.105,0.865]	93.1 [43.3,142.8]	0.1361 [0.0696,0.2027]	154.5 [78.7,230.3]
D-S	0.350 [0.230,0.470]	65.7 [33.8,97.6]	0.1444 [0.0000,0.2942]	236.6 [128.9,344.2]
D-S-w/o-B	0.450 [0.260,0.660]	68.7 [0.0,160.1]	**0.1139** [0.0506,0.1771]	363.8 [263.5,464.2]
DSGN	**0.500** [0.375,0.625]	65.6 [36.7,94.5]	0.1278 [0.0000,0.3201]	**148.8** [81.6,215.9]

Note: D-w/o-B, D-S, and D-S-w/o-B denote DSGN-WO-BETA, DSGN-SCALAR, and DSGN-SCALAR-WO-BETA, respectively. Brackets show 95% confidence intervals.

TABLE A.III
DSGN ACTION-GRANULARITY SENSITIVITY IN TASK SWITCHING

Scenario	Grid	SR↑	$\mathbf{T}_{\mathrm{cap}}$ (s)↓	CR↓	P-Steps↓
3v1	3×3	**0.995** [0.985,1.000]	**24.2** [21.8,26.6]	**0.0000** [0.0000,0.0000]	218.2 [210.0,226.4]
	5×5	0.980 [0.961,0.999]	24.5 [21.9,27.1]	0.0083 [0.0000,0.0169]	**216.0** [209.1,222.9]
6v2	3×3	**0.970** [0.946,0.994]	33.7 [31.7,35.7]	**0.0075** [0.0012,0.0138]	88.8 [84.5,93.1]
	5×5	0.955 [0.926,0.984]	**32.9** [31.2,34.6]	0.0108 [0.0034,0.0182]	**86.2** [82.2,90.2]
9v3	3×3	**0.900** [0.858,0.942]	28.8 [27.2,30.4]	**0.0128** [0.0072,0.0184]	**52.1** [48.6,55.6]
	5×5	0.880 [0.835,0.925]	**27.6** [26.0,29.2]	0.0161 [0.0097,0.0225]	52.9 [49.4,56.4]

Note: Mean with 95% confidence interval in brackets; bold denotes the best mean within each scenario (ties included). The 5×5 grid uses $a_{\mathrm{acc}} \in \{0, \pm 0.2, \pm 0.4\}\ \mathrm{m/s^2}$ and $\omega \in \{0, \pm\pi/12, \pm\pi/6\}\ \mathrm{rad/s}$.